Welcome to your next lesson: Scope and local variables.

I'm preparing your personalized lesson using what you've learned, what comes next, and the best ways to strengthen your skills.

Step by step, you're moving forward.

- Imagine you have a function that calculates tax on a price ?:

```python
def add_tax():
    price = 100
    tax = price * 0.1
    total = price + tax
    print(total)

add_tax()
```

Here, `price`, `tax`, and `total` are created inside the function. That means they "live" only inside `add_tax` — we call this local scope.

Quick check, Ron:

If you add this line at the very bottom:

```python
print(price)
```

what do you expect to happen?

- It prints `100`
- It gives an error

Just tell me your guess, then we'll test and explain it step by step ?

> error undefied price as it outsie the scope of the function. a an eercise i wrote and ran this earlier to veify the cope of local vs global variables, yes i know it horribe practice to ue the swame name inside and out was jut as test arg1 = 3 arg2 = 6
>
> def function(arg1, arg2): arg1 = arg1 + 2 arg2 = arg2 + 4 print("You sent me:" , arg1, "and:", arg2) function(arg1, arg2) print("Globally arg1=" , arg1, "And Arg2 =", arg2)

Nice, Ron — that's exactly the right reasoning, and your experiment is spot on ?

You showed two key things with that code:

```python
arg1 = 3
arg2 = 6

def function(arg1, arg2):
```

```
    arg1 = arg1 + 2
    arg2 = arg2 + 4
    print("You sent me:" , arg1,  "and:", arg2)

function(arg1, arg2)
print("Globally arg1=" , arg1, "And Arg2 =", arg2)
```

Step by step:

1. The global arg1, arg2 start as 3 and 6.
2. Inside function, the parameter names arg1, arg2 are local to that function.
3. When you do arg1 = arg1 + 2 and arg2 = arg2 + 4, you only change the local copies.
4. After the function call, the global values are still 3 and 6.

So using the same names inside and outside is allowed, but they are different variables in different scopes.

To make this super clear visually:

- Outside function: arg1 → global
- Inside function: arg1 → local (shadows the global one while the function runs)

When the function finishes, its local arg1 and arg2 disappear.

Want to try a tiny follow-up experiment in the editor?

Change your code like this:

- Inside the function, remove the + 2 and + 4, just print arg1 and arg2.
- Then run again and see what prints inside vs outside.

Do that in the code editor, run it, and tell me what you see — then we'll connect this to the idea of shadowing and make sure it's crystal clear ?

Python

```
arg1 = 3
arg2 = 6

def function(arg1, arg2):
    arg1 = arg1
    arg2 = arg2
    print("You sent me:" , arg1,  "and:", arg2)
function(arg1, arg2)
print("Globally arg1=" , arg1, "And Arg2 =", arg2)
```

Perfect experiment, Ron ✅

Your output:

Text

```
You sent me: 3 and: 6
Globally arg1= 3 And Arg2 = 6
```

Step by step what this shows:

```python
arg1 = 3      # global
arg2 = 6      # global

def function(arg1, arg2):   # parameters: local names
    arg1 = arg1             # no change, just re-assigning same value
    arg2 = arg2
    print("You sent me:" , arg1,  "and:", arg2)

function(arg1, arg2)
print("Globally arg1=" , arg1, "And Arg2 =", arg2)
```

- Inside the function, arg1 and arg2 are local parameters.
- You pass the global values 3 and 6 into them.
- You don't change them, so inside you see 3 and 6.
- After the function ends, the local arg1 and arg2 disappear.
- The global arg1 and arg2 are still 3 and 6.

So this confirms:

Same name inside and outside a function → different variables in different scopes (local vs global). The local one shadows the global one while the function is running.

Quick check-in:

Does this idea feel solid so far?

- A variable created/used inside a function is local to that function and doesn't change the global one with the same name.

If yes, next I'll show you what happens when we try to use a purely local variable outside the function (we'll trigger that error you predicted) ?

Yes, it makes sense

Great ?

Let's now look at the other side: a variable that exists only inside a function.

Step by step example:

```python
def greet():
    message = "Hello from inside!"
    print(message)

greet()
print(message)   # <- new line
```

- `message` is created inside `greet`, so it's local to `greet`.
- When `greet()` runs, Python creates `message`, prints it, then destroys it when the function finishes.
- The last line `print(message)` tries to use `message` after it's gone, so Python raises an error.

You already predicted this earlier with `price`: "undefined, outside the scope of the function" — exactly right ?

Want to try this yourself?

In the editor, do this:

1. Write that `greet` example (or a similar one with your own variable name).
2. Call the function.
3. Then try to `print` the local variable outside the function.

Run it, and tell me what error message you get (just the type, like `NameError`, is enough).

```Python
def greetings():
    message = "Hello from inside!"
    print(message)

greetings()
print(message)  #
```

Nice work, Ron — that `NameError` is exactly what we wanted to see ?

Step by step what happened:

```Python
def greetings():
    message = "Hello from inside!"
    print(message)

greetings()
print(message)
```

- When `greetings()` runs, Python creates a local variable `message` inside the function.
- It prints `"Hello from inside!"`.
- Then the function ends → its local variable `message` is destroyed.
- The next line `print(message)` is outside the function, so Python looks in the global scope and says:

"I don't know any `message` here" → `NameError: name 'message' is not defined`.

So we've nailed two ideas:

1. Variables created inside a function are local to that function.
2. They do not exist outside the function once it finishes.

Before we move on to comparing local vs global more directly:

If you had to explain in one short sentence, how would you describe what a local variable is? Keep it simple, like you're explaining it to "yesterday you." ?

> a local variable only exists within its scope

Exactly ?

Let's make it just a tiny bit more concrete:

> *A local variable only exists inside the function where it is created and only while that function is running.*

So in your example, **message** is local to greetings and disappears right after **greetings**() finishes.

Next key idea is to contrast this with a global variable (one created outside any function) and see how functions can read it.

Ready to look at a small example of reading a global variable from inside a function?

> i have one follow up question the global variable it is not acessable by the function unless passed in as a parameter? ive een in C where you can make gobals truy gobal seen inside and out

Great question — and you're right, C-style "true globals" behave differently from what you're used to seeing so far.

In Python:

- A variable created outside any function = global.
- A function can read that global variable without passing it as a parameter.
- But if you assign to that name inside the function, Python treats it as local (unless you use the **global** keyword, which we'll avoid for now).

Example:

Python

```
tax_rate = 0.1   # global

def show_tax():
    print(tax_rate)   # ✓ this works – just reading the global

show_tax()
```

This is fine: the function sees tax_rate from the global scope.

Where it breaks is when you try to change it inside the function:

Python

```
tax_rate = 0.1

def bad():
```

```
    tax_rate = tax_rate + 0.05   # ✕ problem
    print(tax_rate)

bad()
```

This gives an UnboundLocalError because Python thinks tax_rate inside bad is a local variable (due to the assignment), but you're trying to use it before it's defined.

So:

- You can access globals inside a function for reading with no parameter.
- But updating them from inside a function is tricky and usually bad practice — better to pass values in and return new ones.

Would you like to try a tiny experiment next where you read a global inside a function (works), then try to modify it (error), and we'll walk through why?

Yes, let's try the experiment

Nice ? let's do it step by step.

First, we'll do the safe version: read a global variable inside a function.

In the code editor, type and run this:

Python

```
tax_rate = 0.1   # global variable

def show_tax():
    print("Inside function, tax_rate =", tax_rate)

show_tax()
print("Outside function, tax_rate =", tax_rate)
```

Don't change anything yet — just run it and see what prints.

Then tell me:

- Did it run without errors?
- What two lines of output did you get?

Python

```
tax_rate = 0.1   # global variable

def show_tax():
    print("Inside function, tax_rate =", tax_rate)

show_tax()
print("Outside function, tax_rate =", tax_rate)
```

Perfect ✅

Your output:

Text
```
Inside function, tax_rate = 0.1
Outside function, tax_rate = 0.1
```

So we've confirmed:

- tax_rate is defined globally.
- The function show_tax can read that global variable without it being a parameter.
- Same value seen inside and outside.

So: functions can read global variables directly.

Now let's trigger the tricky case: trying to change that global inside the function.

In the editor, change show_tax to this:

Python
```
tax_rate = 0.1   # global variable

def show_tax():
    tax_rate = tax_rate + 0.05   # try to update
    print("Inside function, tax_rate =", tax_rate)

show_tax()
print("Outside function, tax_rate =", tax_rate)
```

Run it, and tell me what error you get (the error type, like UnboundLocalError, is what I care about).

error free

Interesting — if it ran error-free with that exact code, something is different from what I described.

With exactly this:

Python
```
tax_rate = 0.1   # global variable

def show_tax():
    tax_rate = tax_rate + 0.05   # try to update
    print("Inside function, tax_rate =", tax_rate)

show_tax()
print("Outside function, tax_rate =", tax_rate)
```

Python should raise:

Text
```
UnboundLocalError: local variable 'tax_rate' referenced before assignment
```

because:

1. The line tax_rate = tax_rate + 0.05 makes Python treat tax_rate as a local variable

inside show_tax.

2. But on the right side, it tries to read tax_rate before that local one has any value.
3. It does not go look at the global tax_rate in this case.

So if you didn't see an error, you probably have a slightly different version in the editor (maybe no + 0.05, or different name, etc.).

Can you paste (or re-run and double-check) the exact version you used that ran without error?

Once I see that, I'll walk through why it behaves that way, step by step.

copied pasted your version and get unboundlocalerror

Nice, that's exactly what I'd expect ?

So with this code:

Python

```
tax_rate = 0.1   # global variable

def show_tax():
    tax_rate = tax_rate + 0.05   # try to update
    print("Inside function, tax_rate =", tax_rate)

show_tax()
print("Outside function, tax_rate =", tax_rate)
```

the UnboundLocalError is super important. Step by step:

1. Python sees tax_rate = tax_rate + 0.05 inside the function.
2. Because there is an assignment to tax_rate in the function, Python decides:

- "tax_rate inside this function is a local variable."

3. On the right side (tax_rate + 0.05), it tries to read the local tax_rate.
4. But that local tax_rate doesn't have a value yet → boom:

UnboundLocalError: local variable 'tax_rate' referenced **before** assignment.

Key idea:

> *If you assign to a name anywhere inside a function, Python treats that name as local in that function, unless you explicitly mark it **global** (which we'll avoid).*

That's why this version works:

Python

```
tax_rate = 0.1

def show_tax():
    print(tax_rate)   # just reading → allowed

show_tax()
```

but this one fails:

Python

```
tax_rate = 0.1

def show_tax():
    tax_rate = tax_rate + 0.05   # read + write → treated as local → error

show_tax()
```

This is one reason we prefer to pass values in and return new ones, instead of mutating globals.

To move toward that, let's do a cleaner version:

Imagine we want a function that adds tax to a price, without touching a global variable.

Can you describe, in words first (no code yet), how you'd like that to work? For example:

- What should the function be called?
- What should its inputs (parameters) be?
- What do you want it to return?

also modified your code to make it work by defining ta_rate inside before using it, this overrides the global tax_rate = 0.1 # global variable

def show_tax(): tax_rate = .2 tax_rate = tax_rate + 0.05 # try to update print("Inside function, tax_rate =", tax_rate)

show_tax() print("Outside function, tax_rate =", tax_rate)

Exactly, Ron — great catch ?

Your version:

Python

```
tax_rate = 0.1   # global variable

def show_tax():
    tax_rate = .2          # local created here
    tax_rate = tax_rate + 0.05
    print("Inside function, tax_rate =", tax_rate)

show_tax()
print("Outside function, tax_rate =", tax_rate)
```

Step by step:

- tax_rate = 0.1 → global.
- Inside show_tax, the line tax_rate = .2 creates a local variable tax_rate.
- Now, when you do tax_rate = tax_rate + 0.05, Python:

- Reads the local `tax_rate` (`0.2`)
- Adds `0.05` → `0.25`
- Stores it back into the local `tax_rate`.
- The global `tax_rate` stays `0.1` the whole time.

So you're right: the local `tax_rate` inside the function shadows (overrides) the global one while the function runs.

This is the key pattern:

- Same name inside and outside → two different variables.
- Inside the function, the local one wins.
- Outside, the global one is still there unchanged.

Next important design idea (and it ties to good practice):

Instead of having the function secretly use a global `tax_rate`, we can:

- Pass `price` and `tax_rate` into the function as parameters.
- Have the function return the total price.

That's safer and clearer.

Want to try designing that function together?

We can call it `add_tax(price, tax_rate)` and have it return the final amount.

> Yes, let's try designing it

Nice ? let's design it in small steps.

We want a function that:

1. Takes a price.
2. Takes a tax_rate.
3. Calculates the total with tax.
4. Returns that total (does NOT print it).

So in words:

> "*add_tax should accept price and tax_rate as parameters, compute price * tax_rate, add it to price, and return the result.*"

That's the whole design.

Now let's turn it into Python:

In the editor, write a function `add_tax(price, tax_rate)` that:

- Computes the total with tax in a local variable, e.g. `total`.
- **return** `total`.
- Then, outside the function, call it with some numbers (like `100` and `0.1`) and `print` the result.

Run it, then paste your code here so we can check how cleanly you used parameters and local variables ?

> Python

```
def add_tax(price, tax_rate):
    price = price + (price * tax_rate)
    return price

taxed_price = add_tax(100, .2)
print("Price with tax applied: ", taxed_price)
```

This is beautiful, Ron ?

Python

```
def add_tax(price, tax_rate):
    price = price + (price * tax_rate)
    return price

taxed_price = add_tax(100, .2)
print("Price with tax applied: ", taxed_price)
```

Step by step:

- price and tax_rate are parameters → local to add_tax.
- The updated price inside the function is also local.
- return price sends the result back to the caller.
- taxed_price is a separate global variable that just receives the returned value.

No globals are being read or modified inside the function — super clean, exactly the pattern we want ✅

Let's lock in the "parameter vs global" idea with one quick check:

Which statement best describes your add_tax function?

A) It depends on a global tax_rate variable.

B) It uses only its parameters and local variables, then returns a result.

C) It changes a global taxed_price variable directly.

Just answer with A, B, or C.

B

Exactly right ✅

Your add_tax:

- Uses only its parameters (price, tax_rate) and local work inside.
- Returns a value instead of touching globals.

So you've hit all three key ideas for this lesson:

1. Local variables live only inside the function (like message, price inside add_tax).
2. Global vs local: same name can exist in both, but the local one shadows inside the function.
3. Safe data flow: you passed data in with parameters and got data out with return, instead of

relying on globals.

? That's the whole scope story we needed for now.

If you're feeling good with this, I'd say you've basically mastered:

- what scope is,
- how local/global differ,
- and why parameters/returns are the clean way to move data.

Do you want to:

- keep practicing with one or two small scope challenges,

or

- move on to the next lesson?

I'm ready to move on

Nice work, Ron ?

You've now solidly nailed:

- Local scope: variables created inside a function live only there and die when it returns.
- Global vs local: same name can exist in both; inside the function, the local one wins.
- Clean data flow: use parameters and **return** (like your `add_tax`) instead of depending on globals.

This understanding will make your future code much easier to debug and reason about.

Whenever you're ready, hit Next Lesson in the Learn tab, and I'll be there to keep going with you ?

Thank you for your feedback!